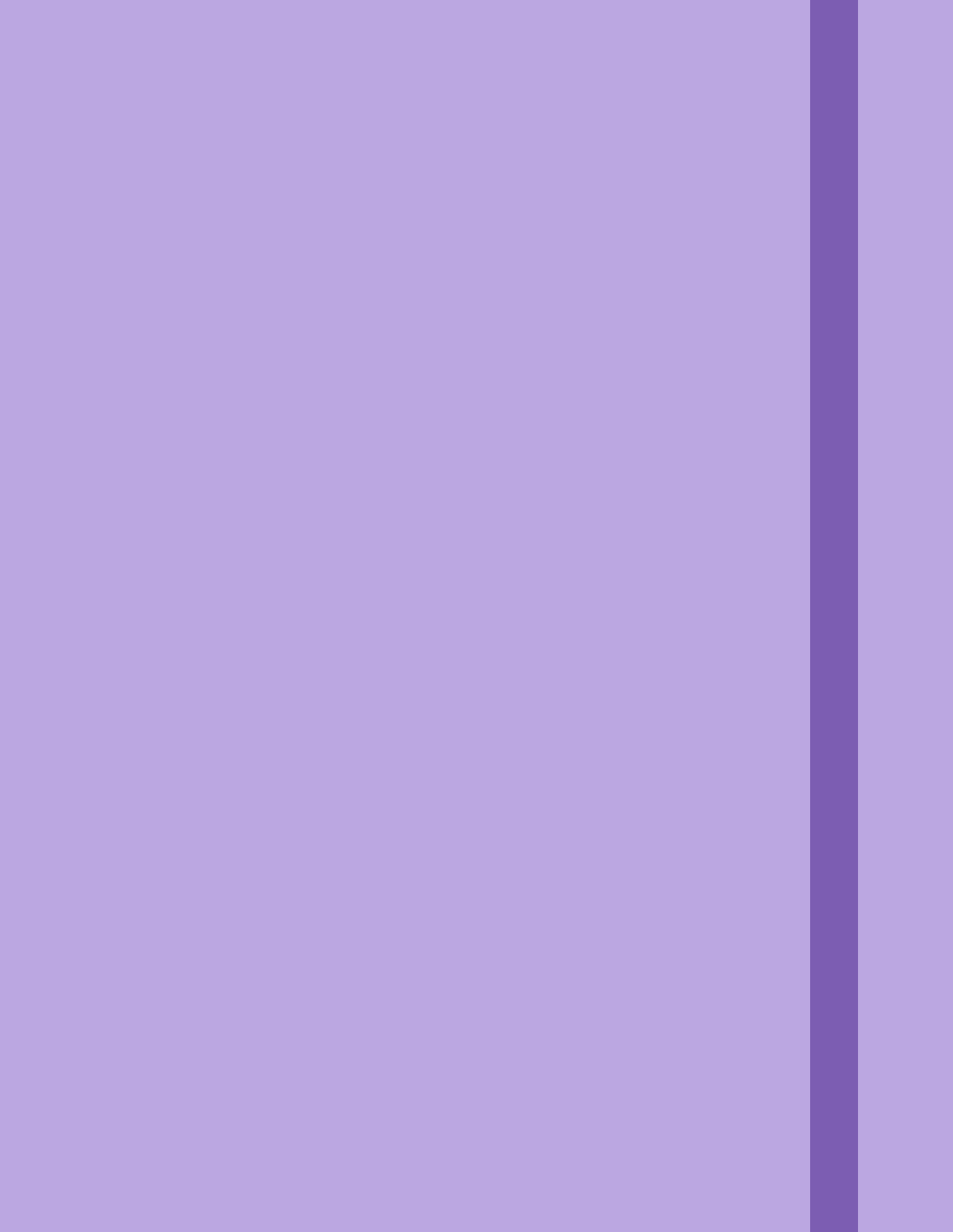




Valid Sudoku

Valid Sudoku

36. Valid Sudoku

Medium Topics Companies

Determine if a 9×9 Sudoku board is valid. Only the filled cells need to be validated according to the following rules:

1. Each row must contain the digits 1-9 without repetition.
2. Each column must contain the digits 1-9 without repetition.
3. Each of the nine 3×3 sub-boxes of the grid must contain the digits 1-9 without repetition.

Note:

- A Sudoku board (partially filled) could be valid but is not necessarily solvable.
- Only the filled cells need to be validated according to the mentioned rules.

Example 1:

5	3		7					
6		1	9	5				
	9	8			6			
8			6				3	
4		8	3				1	
7			2				6	
	6				2	8		
		4	1	9			5	
			8		7	9		

Input: board =

```
[["5","3","","7","","","","",""],
 ["6","","1","9","5","","","",""],
 ["","9","8","","","6","","",""],
 ["8","","","6","","","","3",""],
 ["4","","8","3","","","","1",""],
 ["7","","","2","","","","6",""],
 ["","6","","","","2","8","",""],
 ["","","4","1","9","","","5",""],
 ["","","","8","","7","9","",""]]
```

Output: true

For this solution we use a hashset

*

hash set - Python Dictionary only stores Unique pair value like: {"key": "value"}

Here there will be 3 dictionaries

1. For Row: Key: Row-Index, Value: Value at Index
2. For Column: Key: Column-Index, Value: Value at Index
3. For 3×3 Square: Key: Square-Index, Value: Value at Index

To get Square index we do $r // 3, c // 3$

For Example (2,1) ←

$$\frac{2}{3}, \frac{1}{3} = 0, 0$$

Integer

$$\frac{6}{3}, \frac{8}{3} = 2, 2$$

	0	1	2	3	4	5	6	7	8
0	0,0			0,1			0,2		
1									
2	2,0			1,1			1,2		
3									
4	2,0			2,1			2,2		
5									
6									
7									
8									

Only One iteration through the matrix required in that iteration we have to:

1. Check if value present in 3 dictionary
2. If not update dictionary

</> Code

Python3 ▾ Auto

```
1 class Solution:
2     def isValidSudoku(self, board: List[List[str]]) -> bool:
3         row = collections.defaultdict(set)
4         col = collections.defaultdict(set)
5         box = collections.defaultdict(set)
6
7         for r in range(9):
8             for c in range(9):
9                 if board[r][c] == ".":
10                     continue
11                 if (board[r][c] in row[r] or
12                     board[r][c] in col[c] or
13                     board[r][c] in (box[(r//3, c//3)])):
14                     return False
15                 row[r].add(board[r][c])
16                 col[c].add(board[r][c])
17                 box[(r//3, c//3)].add(board[r][c])
18         return True
19
20
```

Accepted 507 / 507 testcases passed

Jatin Khiyani submitted at Dec 14, 2025 16:01

Editorial

Solution

Runtime

3 ms | Beats 81.38%

Analyze Complexity

Memory

17.96 MB | Beats 24.50%

